An example of constructor & Method :

```
class Test
{
    void main()
    {
        Employee e1 = new Employee();
        e1.Employee();
    }
}
class Employee
{
    public Employee()  //No return type so, It is constructor
    {
        IO.println("It is a Constructor");
    }

    public void Employee()  //Here we have return type so, It is method
    {
        IO.println("It is a Method");
    }
}
```

Types of Constructor :
----
1) Implicit OR Compiler added Constructor
2) Explicit OR User-defined Constructor:
----

1) Implicit OR Compiler added Constructor

It is also known as default no argument constructor.
It is automatically provided by java compiler, if a user does not provide OR write any type of constructor.
It is provided by compiler to give the support of **object creation** in java without writing explicit OR user-defined constructor.

Explicit OR User-defined Constructor

The constructor which is explicitly written by user in the class is called Explicit OR User-defined constructor.
It is divided into two types:
a) No Argument OR Parameter less OR Non parameterized OR Zero Argument Constructor
b) Parameterized Constructor

a) No Argument OR Parameter less OR Non parameterized OR Zero Argument Constructor
If a constructor is written by user **without any parameter**, then it is called No Argument Constructor.
It is used to provide the default value for the non-static fields.
Actually, Internally It **will copy the default values** provided by new keyword.

Example:
```
public class Student
{
private int roll;
private String name;
public Student() //No Argument user-defined constructor
{
roll = 0;
name = null;
}
}
```

b) Parameterized Constructor:

If we pass one or more parameters to the constructor then It is called Parameterized constructor.

It is mainly used to initialized the non-static fields with parameter variable using user values. [Replacing the default value with user value]

Example:
```
public class Employee
{
private int id;
private String name;
public Employee (int id, String name) //Parameterized constructor
{
this.id = id;
this.name = name;
}
}
```

WAP to initialize the non static fields with parameter variables using parameterized constructor :

```
package com.ravi.parametrized_constructor;

public class PersonDemo
{
    public static void main(String[] args)
    {
       Person scott = new Person(1, "Scott", 5.8, 23, "Ameerprt");
       IO.println(scott);
    }
}
class Person
{
    private int id;
    private String name;
    private double height;
    private int age;
    private String address;

    public Person(int id, String name, double height, int age, String address)
    {
        this.id = id;
        this.name = name;
        this.height = height;
        this.age = age;
        this.address = address;
    }

    @Override
    public String toString()
    {
        return "Person [id=" + id + ", name=" + name + ", height=" + height + ", age=" + age +
", address=" + address
                + "]";
    }

}
```

WAP to initialize the non static fields with parameter variables using parameterized constructor
----
with data validation

```
package com.ravi.parametrized_constructor;

public class DogDemo
{
   void main()
   {
       String name = IO.readln("Enter Dog Name :"); //null   ---> "null"
       int age = Integer.parseInt(IO.readln("Enter Dog Age :"));
       double height = Double.parseDouble(IO.readln("Enter Dog height :"));
       String color = IO.readln("Enter Dog Color :");

       Dog d1 = new Dog(name, age, height, color);
       IO.println(d1);
   }
}

class Dog
{
    private String name;
    private int age;
    private double height;
    private String color;

    public Dog(String name, int age, double height, String color)
    {
        if(name==null || name.isBlank() || name.equals("null"))
        {
            System.err.println("Dog name cannot be empty OR blank");
            System.exit(0);
        }

        if(age <=0)
        {
            System.err.println("Dog age cannot be 0 OR negative");
            System.exit(0);
        }

        if(height <=0)
        {
            System.err.println("Dog height cannot be 0 OR negative");
            System.exit(0);
        }

        if(color==null || color.isBlank() || color.equals("null"))
        {
            System.err.println("Dog color cannot be empty OR blank");
            System.exit(0);
        }

        this.name = name;
        this.age = age;
        this.height = height;
        this.color = color;
    }

    @Override
    public String toString()
    {
        return "Dog [name=" + name + ", age=" + age + ", height=" + height + ", color=" +
color + "]";
    }

}
```

How to write setter & getter in Java :

Example :

```
public class Product
{
   private String name;
   private double price;

   //Parameterized Constructor to initialize the non static fields

   //Setter & getter for name field
   public void setName(String name)   //setter
   {
       this.name = name;
   }

   public String getName()
   {
      return this.name;
   }

   public void setPrice(double price)
   {
      this.price = price;
   }

   public double getPrice()
   {
      return this.price;
   }
}
```